DLMed Project 2: Q&A with Claude

Oxford-IIIT Pet multi-task review · 2026-10-16 · questions and final answers only

QUESTION 1

This is a new project.
Workspace: `C:\workspace\vscode\DL-in-Medical-Image-Project-2`
Dataset: `C:\workspace\vscode\oxford-iiit-pet`

This is a image deep learning project for multi-task learning on oxford-iit-pet dataset. The task is to train an image segmentation and classification model on pet images to:

1) Segment the pet from the background.
2) Rough classification on two classes (dog/cat)
3) Per-species classification. (2 and 3 are seperate heads and does not depend on each other.)

My team has trained the model. I have downloaded the checkpoint file at `C:\workspace\vscode\DL-in-Medical-Image-Project-2\checkpoints\best_model_resnet50.pth`. My task is to make sure that the training session went as intended as per the rubrics before handing out a sort of guideline for the presentation team. (See more at `C:\workspace\vscode\DL-in-Medical-Image-Project-2\docs\Project 2 Instruction.pdf`)
Right now, we need to:

1) Create a `.gitignore` file.
Keep the zip files as is.

2) Recheck the work for any implementation flaws, and against the rubrics.
Also see: https://de.aipass.net/share/46bf3f41e67e4bd4bb837536c206f451 But let's suppose the flaws listed there are taken care of.

3) Add contents to the overview html file.
Add the listed context, components. Avoid editing the old contents and use the same schematics.

3.1) Data Understanding
Do the reserach. List out all breeds contained in the detaset. Point out their highlights/distinctive features.

3.2) Component Deepdive
The current overview covers the full pipeline. But I think we need to get in details on our decision choices. Especially, how it compares against the course materials. I handpicked the most relevant handouts for the project here: `C:\workspace\vscode\DLMed`

Specifically, I want the doc to point out what is behind that exact conponent choice (and why not other.) In readme, "จุดเด่น" section, there is a list of the techniques/metrics used. Explain those.

3.3) Noteworthy Questions
After the presentation, we will receive a question regarding the work. Predict the question we may get then and a helpful answer to each.

3.4) An inference interface.
A self-serving interface with a guide and requirement. The interface may have two modes: inference and

evaluation.

Inference: Select an image from local storage. Show the inference result.
Evaluation: Select a folder. Show the metrices.

You also have the liberty to decouple the html file. Treat `index.html` as the landing page and make sure it works as a static file without model serving. Item 3.1, 3.2, and 3.4 can be its own page navigated via navbar, while item 3.3 can be appended at the main page.

With this shift, the Jupyter file now works as code tour. And the web now works as an interactive document.

4) Update `README.md`
Document the changes. If possible and appropriate, edit the existing contents and write the incoming ones in github-compatible mermaid diagram. Avoid using TeX.

Use Thai in documents. Do not commit yet.

ANSWER

Written for: the presentation team (the Thai web pages and `docs/REVIEW.md`); this summary is for you.

All four tasks are done and nothing is committed. Your dataset folder was empty, so I downloaded the official Oxford archives into `C:\workspace\vscode\oxford-iiit-pet` (~810 MB). I also created a `.venv` in the repo (CPU PyTorch + ONNX), which is gitignored.

1. `.gitignore`

It excludes venvs, the dataset, `*.pth` checkpoints, ONNX files, `outputs/` and caches. The zips in `examples/` are still tracked, and the link `.txt` files in `checkpoints/` and `data_cache/` are kept. `Inference_Image_Data.zip` was already deleted before I started; I didn't touch it.

2. Review (docs/REVIEW.md)

The training went as intended. I re-ran the checkpoint on val and test. It is the Balanced config at epoch 10. Every test number reproduces exactly, except breed accuracy (91.66% vs 91.69%, one image, from CPU vs GPU). The validation score also matches the training log.

Two things need attention before the slides:

- **The notebook's Sphynx conclusion is wrong.** Section 4.1 says Sphynx → Chihuahua (12 images) shows the two heads don't agree. In 10 of those 12, the species head also says "dog", so both heads are wrong together. Across the whole test set the heads disagree on only 10 images. The old Confusion Matrix section in `index.html` repeats this claim; I left it unedited as you asked and flagged it in the Q&A and the review.
- **"Compare at least 3 forms" is the weakest rubric point.** The three variants differ only in λ, and the checkpoints for the other two aren't in the download, so only Balanced can be verified. The cheapest fix is a ResNet-34 run, which the config already supports, or single-task baselines.

I also found 8 smaller implementation issues (none change the reported numbers). Two matter most:

- Checkpoint selection is noisy: epoch 10 wins because of one breed-F1 spike on a small val set.
- `inference.py` ignores EXIF orientation, so phone photos can come in rotated during a live demo.

3. Web docs

- **`index.html`:** the existing text is unchanged. I moved its CSS and images into `web/assets/`, added a navbar and a dark/light toggle, and appended a Q&A section with 25 questions and suggested answers.
- **`web/data.html`:** dataset stats, groups of look-alike breeds with measured confusion counts, and a card for each of the 37 breeds (test-set photo, distinctive features, per-breed F1).
- **`web/components.html`:** 10 components, each with what we chose, why, the DLMed handout pages that support it, the alternatives we didn't pick, and the question a grader is likely to ask. All 6 "จุดเด่น" items from the README are explained there.
- **`web/inference.html`:** single-image inference and folder evaluation, running in the browser. Setup is `python export_onnx.py`, then `python -m http.server 8000`. On 40 test images the browser results match PyTorch to 4 decimal places.
  - onnxruntime-web 1.20.1 gave wrong results on WebGPU when processing more than one image at a time. I moved to 1.22.0, which fixes it.
- All pages work as static files and fit phone width.

4. README

- The architecture, data-split and training-loop diagrams are now Mermaid; I checked that all 6 diagrams render.
- Updated the project tree and test results, and fixed the wrong Albumentations badge and the broken example paths.
- New sections: web docs, browser demo, review summary, and a changelog.

The Obsidian vault has a new `wiki/DLMed-Project-2/` topic (Introduction, Progress), plus entries in `index.md` and `log.md`.

QUESTION 2

Review: I am now downloading the other varients with different lambda. I can let you test with them if the downdloading is finished.

Data Understanding:
- The breed cards have too much text upfront. I suggest you do like a simple name/f1 overlay on small catalog. The details can show up at the side when clicked.
- It can be helpful to have the other species apart from the ones in the dataset as the reference. Cite your sources.
- Can it be better visualized?

Model Playground:
- The actual playground should be the first thing to show up.

Web docs as a whole:
- Model playground should be highlighted, and on the rightmost on navbar.
- Provide clickable external URLs.
- I thought that the question section would be realitively short. However, it does warrant for its own page. You can move the question part from the index to its own page.
- I told you to avoid TeX, but I should've been more clear. The reason is that on GitHub Mobile, TeX couldn't render properly within markdown files. So what I meant is to avoid TeX in markdowns. For the web, though, you can use it (especially for the deep-dive.)

ANSWER

Everything from your feedback is in, and nothing is committed. I checked every page in headless Chrome in dark and light themes and at phone width. The playground still matches PyTorch end to end.

Data page (`web/data.html`)

- **Catalog:** small photo tiles showing only the name, F1 (red if under 80%) and a cat/dog dot. Clicking a tile opens a side panel with:
  - the six scores (F1, recall, precision, segmentation IoU, test and train image counts);
  - what to look for in a photo, and look-alike breeds you can click through;
  - which breeds the model confuses it with, and source links.

  It closes with Esc, the × or a click outside. A link like `data.html#breed=ragdoll` opens a breed directly.
- **Breeds outside the dataset:** 23 look-alikes the model has never seen (Akita, Husky, AmStaff, Korat, Savannah, ...), including 4 Thai breeds. Each card says which dataset breed it would probably be mistaken for. These look-alikes are my judgment from appearance and haven't been run through the model, except Akita, which the notebook already tested. The page says so.
- **Sources:** every breed links to a breed registry (AKC or UKC for dogs, CFA for cats) and English Wikipedia. I checked all 116 links.
- **Charts** (each with a hover tooltip and a "view as table" option):

- image counts per split, cat vs dog;
- image size and aspect-ratio histograms;
- how the mask pixels split between background, pet and boundary;
- per-breed classification score vs segmentation score;
- the 10 most-confused breed pairs.

The size chart turned up something useful for the slides: 83% of images are only about 500px on the long side, so shrinking to 224 leaves about 45% of the resolution per side.

Playground: the model loader and the Inference/Evaluation tabs now come first, right under a short header. Setup instructions moved below them.

Whole site

- The navbar now has a highlighted **▶ ทดลองโมเดล** button at the far right on every page; the theme toggle is a small icon next to it.
- The questions moved to their own page, `web/qa.html`, with a contents list per group and an open-all/close-all button. They're removed from `index.html`.
- External links are clickable throughout and marked ↗, including every paper on the deep-dive page (arXiv or DOI).
- Formulas render with KaTeX on the deep-dive and Q&A pages. The README still has no TeX, and its 6 Mermaid diagrams still render.

The chart colours pass a colour-blind safety check in both themes. The one exception is the mask colours (purple/teal/yellow): I kept them so they match the mask figures, and they always have text labels next to them.

λ variants: once the Seg-focused and Cls-focused checkpoints finish downloading, tell me where they are. I'll re-run each one on the test set (about 11 minutes each on CPU) and check the numbers against the notebook's comparison table. Then I'll update `docs/REVIEW.md` and the comparison claims in the pages.

> **QUESTION 3**
> The checkpoints should be ready.

ANSWER

I tested both variants, and the notebook's comparison table (cell 6) doesn't match these checkpoints. Its conclusion doesn't hold for them either.

Both checkpoints are set up correctly: same config as Balanced except for λ. I ran all three models on the same val and test images.

Variant	Epoch picked	Val score	Test mIoU	Test breed accuracy	Notebook says (mIoU / breed F1)
Balanced	10	**0.8863**	**0.8266**	91.66%	0.8266 / 0.9154 ✅
Seg-focused	28	0.8832	0.8234	92.64%	0.8285 / 0.8942 ❌
Cls-focused	27	0.8842	0.8222	**93.08%**	0.8094 / 0.9168 ❌

"Val score" is the training script's selection score, (mIoU + breed F1) / 2 on val.

- **The story flips.** The notebook says "push a task's weight up and that task improves a little while the other drops a lot." With these files, both variants classify breeds better than Balanced and segment slightly worse. That includes Seg-focused, which doubled the segmentation weight.
- **The differences are real but uneven.**
  - Breed accuracy: +1.0 and +1.4 points, which is significant (McNemar test $p = 0.006$ and $p < 0.001$).
  - mIoU: −0.003 and −0.004. Statistically real (the 95% bootstrap interval excludes 0) but tiny in practice.
- **λ isn't the only thing that changed.** Balanced's checkpoint is from epoch 10; the other two are from epochs 27–28, after the learning rate had decayed. So most of their classification edge probably comes from training longer, not from λ. The comparison can't separate the two yet.
- **Balanced is still the right main model.** It wins on val, which is the selection rule fixed before training. Switching to Cls-focused because its test numbers look better would mean choosing the model with the test set.

I updated `docs/REVIEW.md` with the full comparison, significance tests and a to-do list. I also corrected the λ table on the deep-dive page, rewrote the affected Q&A answers, and added a verified results table and a warning to the README.

Decisions for you:

- **Ask the training team where the cell-6 numbers came from.** They might be from an earlier training run. Or `evaluate.py` was used, which always loads `best_model_resnet50.pth` unless it's edited.
- **Should I correct the old content?** I haven't touched the notebook's cell 6 or the old "ทดลอง 3 แบบ" section in `index.html`, since you asked me to leave old content alone. Both still show the old numbers and the old conclusion. I can update them, or the team can fix them at the source.
- **A fair comparison needs one more run per model:** evaluate all three at the same epoch, for example the last one. The current checkpoints only keep the best epoch, so that means retraining or saving the final weights.

The comparison script is still in my scratch folder. I can add it to the repo as a script if the team wants to re-run it.

QUESTION 4

Commit and push.

ANSWER

I committed everything as `625b755` on `main` and pushed it to origin. The commit contains all of rounds 1–3: `docs/REVIEW.md`, `index.html`, the five `web/` pages, `export_onnx.py`, the README and `.gitignore`.

- **One zip had gone missing.** `Inference_Image_Data.zip` disappeared when the files were moved into `examples/` and would have gone into the commit as a deletion. I restored it from git first. All three zips are now in `examples/`, and git records the moves as renames, so nothing was lost.
- **Not pushed:** `.venv`, the checkpoints and the 186 MB `web/model/pet_multitask.onnx` are all excluded by `.gitignore`. On a fresh clone, the playground has no model until someone runs `export_onnx.py` or picks the ONNX file by hand.
- **Still unchanged:** the notebook's cell 6 and the old "ทดลอง 3 แบบ" section in `index.html` still show the old variant numbers. Tell me if you want them corrected in a follow-up commit.

I also updated the Obsidian Progress note, the vault log and my memory note.

QUESTION 5

How can python just serve on localhost?

ANSWER

Python includes a small web server in its standard library, the `http.server` module, so there's nothing to install. Run it from the repo root:

```
cd DL-in-Medical-Image-Project-2
python -m http.server 8000 --bind 127.0.0.1
```

Then open `http://localhost:8000/` for the landing page or `http://localhost:8000/web/inference.html` for the playground. Press Ctrl+C to stop it.

How it works:

- `-m http.server` runs the module as a script. It opens a socket on port 8000 and answers HTTP GET requests.
- It maps each URL path straight to a file under the current folder. `/web/inference.html` returns `./web/inference.html`.
- It sends a `Content-Type` based on the file extension. With no file name in the URL, it serves `index.html` or a directory listing.
- `--bind 127.0.0.1` means only your own machine can connect. Without it, it listens on all network interfaces, so anyone on the same Wi-Fi could browse the folder. Recent Python versions often bind dual-stack by default, so pass `--bind` explicitly to be sure.

Why the playground needs it: if you open the page by double-clicking (`file://`), the browser blocks `fetch()` of local files for security. The page then can't load `web/model/pet_multitask.onnx`. Over `http://localhost`, the same request is an ordinary download, so it works. WebGPU also generally requires a secure context, and `localhost` counts as one.

Caveats:

- It's a development server, not a production one. It has no HTTPS, it's mostly single-threaded and has only basic security checks. That's fine for a demo on your own machine, but don't expose it to the internet.
- The ONNX model isn't in git (it's 186 MB). Run `python export_onnx.py` first so `web/model/pet_multitask.onnx` exists. Otherwise, choose the file by hand in the playground.
- You can use any free port. If 8000 is taken, try `python -m http.server 8080`.